```cpp
#pragma once
#include <iostream>
#include "matrix.hpp"
#include "capaMultiHeadAtencion.hpp"
#include "capaDensa.hpp"
#include "LayerNorm.hpp"

class BloqueTransformer {
private:
    capaMultiHeadAtencion atencion;
    LayerNorm norm1;
    LayerNorm norm2;

    capaDensa ffn1;
    capaDensa ffn2;

    Matriz X_cache;
    Matriz xNorm1_cache;
    Matriz res1_cache;

public:
    BloqueTransformer(size_t dimModel, size_t numHeads, size_t dimFFN)
        : atencion(dimModel, numHeads),
          norm1(dimModel),
          norm2(dimModel),
          ffn1(dimModel, dimFFN, tangenteHiperbolica, derivadaTanHiper),
          ffn2(dimFFN, dimModel, tangenteHiperbolica, derivadaTanHiper)
    {}

    Matriz forward(const Matriz& X) {
        X_cache = X;

        // 1. MHA + Conexión Residual
        Matriz atnOut = atencion.forward(X);
        res1_cache = X + atnOut;
        xNorm1_cache = norm1.forward(res1_cache);

        // 2. FFN por columna
        Matriz ffnOut(X.filas(), X.columnas());
        for (size_t col = 0; col < xNorm1_cache.columnas(); ++col) {
            Matriz tokenVec(xNorm1_cache.filas(), 1);
            for (size_t fila = 0; fila < xNorm1_cache.filas(); ++fila) {
                tokenVec(fila, 0) = xNorm1_cache(fila, col);
            }

            Matriz h = ffn1.forward(tokenVec);
            Matriz y = ffn2.forward(h);

            for (size_t fila = 0; fila < xNorm1_cache.filas(); ++fila) {
                ffnOut(fila, col) = y(fila, 0);
            }
        }

        // 3. Segunda Conexión Residual
        Matriz res2 = xNorm1_cache + ffnOut;
        return norm2.forward(res2);
    }

    Matriz backward(const Matriz& gradSalida, double lr, int t) {
        Matriz dFFN(gradSalida.filas(), gradSalida.columnas());
        for (size_t col = 0; col < gradSalida.columnas(); ++col) {
            Matriz gradToken(gradSalida.filas(), 1);
            for (size_t fila = 0; fila < gradSalida.filas(); ++fila) {
                gradToken(fila, 0) = gradSalida(fila, col);
            }
```

```
            Matriz dh2 = ffn2.backward(gradToken, lr, t);
            Matriz dy1 = ffn1.backward(dh2, lr, t);

            for (size_t fila = 0; fila < gradSalida.filas(); ++fila) {
                dFFN(fila, col) = dy1(fila, 0);
            }
        }

        Matriz dRes1 = gradSalida + dFFN;
        Matriz dAtn = atencion.backward(dRes1, lr, t);

        return dRes1 + dAtn;
    }

    void guardarBinario(std::ofstream& out) const {
        atencion.guardarBinario(out);
        ffn1.guardarBinario(out);
        ffn2.guardarBinario(out);
    }

    void cargarBinario(std::ifstream& in) {
        atencion.cargarBinario(in);
        ffn1.cargarBinario(in);
        ffn2.cargarBinario(in);
    }
};
```